

Busca em vetor ordenado

[“Binary search is to algorithms what a wheel is to mechanics: It is simple, elegant, and immensely important.”](#)

— Udi Manber, *Introduction to Algorithms*

[“Binary search is a notoriously tricky algorithm to program correctly. It took seventeen years after its invention until the first correct version of binary search was published!”](#)

— Steven Skiena, *The Algorithm Design Manual*

Este capítulo estuda o seguinte problema de busca: encontrar um dado número em um vetor ordenado de números. Mais especificamente, dado um inteiro x e um vetor [crescente](#) $v[0..n-1]$ de inteiros,

encontrar um índice m tal que $v[m] == x$.

É claro que nem toda [instância](#) desse problema tem [solução](#). Considere, por exemplo, as instâncias em que n vale 0 e aquelas em que $v[3] < x < v[4]$.

Sumário:

- [O problema da busca](#)
- [Busca sequencial](#)
- [Busca binária](#)
- [A função buscaBinaria está correta?](#)
- [Desempenho da busca binária](#)
- [Versão recursiva da busca binária](#)

O problema da busca

Começemos por adotar uma formulação mais geral e mais interessante do problema: encontrar o lugar no vetor onde x *deveria* estar. Mais exatamente, dado x e um vetor crescente $v[0..n-1]$, queremos *encontrar um índice j tal que*

$$v[j-1] < x \leq v[j].$$

Para obter uma solução do problema original, basta comparar x com $v[j]$.

Para tirar o máximo proveito dessa formulação do [problema](#), devemos permitir que a solução j assumira os valores 0 e n . Nesses dois casos, a expressão $v[j-1] < x \leq v[j]$ deve ser interpretada com bom senso: se $j == 0$, a expressão se reduz a $x \leq v[0]$, pois $v[-1]$ não está definido; se $j == n$, a expressão se reduz a $v[n-1] < x$, pois $v[n]$ não está definido. Tudo se passa como se nosso vetor tivesse um componente imaginário $v[-1]$ com valor $-\infty$ e um componente imaginário $v[n]$ com valor $+\infty$.

No exemplo a seguir, se x vale 555, a solução j do problema é 4; se x vale 800, a solução é 8; e se x vale 1000, a solução é 13.

0														$n-1$	
111	222	333	444	555	555	666	777	888	888	888	999	999	999	999	999

Vamos examinar dois algoritmos para o problema: um óbvio mas lento e outro sofisticado e muito mais rápido.

Exercícios 1

1. Escreva uma função que decida se um vetor $v[0..n-1]$ está em ordem crescente. Depois, critique o código a seguir.

```
int verifica (int v[], int n) {
    int anterior = v[0], sim = 1;
    for (int i = 1; i < n && sim; ++i) {
        if (anterior > v[i]) sim = 0;
        anterior = v[i];
    }
    return sim;
}
```

2. Critique a seguinte formulação do problema de busca: dado x e um vetor crescente $v[0..n-1]$, encontrar um índice j tal que $v[j-1] \leq x \leq v[j]$.
3. Critique a formulação do problema de busca construída em torno da expressão $v[j-1] < x < v[j]$.

Busca sequencial

Começamos com um algoritmo óbvio, que examina um a um todos os elementos do vetor. Eis uma implementação do algoritmo:

```
// Esta função recebe um inteiro x e um vetor
// crescente v[0..n-1] e devolve um índice j
// em 0..n tal que  $v[j-1] < x \leq v[j]$ .

int
buscaSequencial (int x, int n, int v[]) {
    int j = 0;
    while ( $j < n$  &&  $v[j] < x$ )
        ++j;
    return j;
}
```

Quantas iterações a função faz? Ou melhor, quantas comparações faz entre x e elementos de v ? No pior caso, x é comparado com cada elemento do vetor e portanto o número total de comparações é n . Assim, por exemplo, se o tamanho do vetor for multiplicado por 100, o número de comparações também será multiplicado por 100.

O consumo de tempo da função é proporcional ao número de comparações que envolvem x , e portanto proporcional a n no pior caso.

É possível resolver o problema em menos tempo? É possível resolver o problema sem comparar x com cada um dos elementos do vetor? A resposta é afirmativa, como veremos a seguir.

Exercícios 2

1. INVARIANTES. Quais são os [invariantes](#) do processo iterativo na função [buscaSequencial](#)? Use os invariantes para mostrar que a função está correta. [[Solução](#)]

2. Discuta a seguinte versão da função `buscaSequencial`:

```
int buscaSeq2 (int x, int n, int v[]) {
    int j;
    for (j = 0; j < n; ++j)
        if (x <= v[j]) break;
    return j; }
```

3. Critique a seguinte versão da função `buscaSequencial`:

```
int buscaSeq3 (int x, int n, int v[]) {
    int j = 0;
    while (v[j] < x && j < n) ++j;
    return j; }
```

4. Discuta a seguinte versão recursiva da função `buscaSequencial`:

```
int buscaSeq4 (int x, int n, int v[]) {
    if (n == 0) return 0;
    if (x > v[n-1]) return n;
    return buscaSeq4 (x, n-1, v); }
```

5. Escreva uma versão da função `buscaSequencial` que satisfaça uma especificação diferente da nossa: dado x e um vetor crescente $v[0..n-1]$, encontrar um índice j tal que $v[j] \leq x < v[j+1]$.

Busca binária

Existe um algoritmo muito mais rápido que a busca sequencial. Ele é análogo ao método que se usa para encontrar um nome em uma lista telefônica antiga, aquela que parece um livro. É claro que a ideia só funciona porque o vetor está *ordenado*.

```
// Esta função recebe um inteiro x
// e um vetor crescente v[0..n-1]
// e devolve um índice j em 0..n
// tal que v[j-1] < x <= v[j].

int
buscaBinaria (int x, int n, int v[]) {
    int e = -1, d = n; // atenção!
    while (e < d-1) {
        int m = (e + d)/2;
        if (v[m] < x) e = m;
        else d = m;
    }
    return d;
}
```

Simple e limpo! Os nomes das variáveis não foram escolhidos ao acaso: e lembra “esquerda”, m lembra “meio” e d lembra “direita”. O resultado da divisão por 2 na expressão $(e+d)/2$ é automaticamente [truncado](#), pois as variáveis são do tipo `int`. Por exemplo, se e vale 6 e d vale 9, a expressão $(e+d)/2$ vale 7.

0				e				d				n-1	
111	222	333	444	555	555	666	777	888	888	888	999	999	999

A ideia da busca binária (= *binary search*) é um verdadeiro [ovo de Colombo](#). Essa ideia é o ponto de partida de algoritmos eficientes para muitos problemas.

Exercícios 3

1. Para evitar valores de índices fora do intervalo $0..n-1$, poderíamos trocar a linha “ $e = -1; d = n$ ” da função `buscaBinaria` pelas três linhas a seguir. Discuta essa variante do código.

```
if (v[n-1] < x) return n;
if (x <= v[0]) return 0;
// agora v[0] < x <= v[n-1]
e = 0; d = n-1;
```

2. Responda as seguintes perguntas sobre a função `buscaBinaria`. Que acontece se “`while (e < d-1)`” for trocado por “`while (e < d)`”? ou por “`while (e <= d-1)`”? Que acontece se trocarmos “`(e + d)/2`” por “`(e + d - 1)/2`” ou por “`(e + d + 1)/2`” ou por “`(d - e)/2`”? Que acontece se “`if (v[m] < x)`” for trocado por “`if (v[m] <= x)`”? Que acontece se “`e = m`” for trocado por “`e = m+1`” ou por “`e = m-1`”? Que acontece se “`d = m`” for trocado por “`d = m+1`” ou por “`d = m-1`”?
3. Suponha que $n = 9$ e $v[i] = i$ para cada i . Execute `buscaBinaria` com vários valores de x . Repita o exercício com $n = 15$ e vários valores de x . Repita o exercício com $n = 14$ e $x = 9$.
4. Execute a função `buscaBinaria` com $n = 16$. Quais os possíveis valores de m na primeira iteração? Quais os possíveis valores de m na segunda iteração? Na terceira? Na quarta?
5. Na função `buscaBinaria`, é verdade que m está em $0..n-1$ sempre que a instrução `if (v[m] < x)` é executada? (Note que e e d podem não estar em $0..n-1$.)
6. Confira a validade da seguinte afirmação: quando $n+1$ é uma potência de 2, a expressão $(e + d)$ é divisível por 2, quaisquer que sejam v e x .

A função buscaBinaria está correta?

Para entender a função `buscaBinaria`, basta [verificar](#) que no início de cada repetição do `while`, imediatamente antes da comparação de e com $d-1$, vale a relação

$$v[e] < x \leq v[d].$$

Essa relação é, portanto, [invariante](#). (O algoritmo foi *construído* a partir desse invariante!) Note a semelhança entre o invariante e o objetivo $v[j-1] < x \leq v[j]$ que estamos perseguindo. O invariante vale, em particular, no início da *primeira* iteração: basta imaginar que $v[-1]$ vale $-\infty$ e $v[n]$ vale $+\infty$.

No início de cada iteração, em virtude do invariante, temos $v[e] < v[d]$ e portanto $e < d$, uma vez que o vetor é crescente. No início da última iteração, temos $e \geq d-1$ e portanto $e == d-1$. A relação invariante garante agora que, ao devolver d , a função está se comportando como prometeu!

Em cada iteração temos $e < m < d$ (por quê?). Logo, tanto $d - m$ quanto $m - e$ são estritamente menores que $d - e$. Portanto, a sequência de valores da expressão $d - e$ é [estritamente decrescente](#). É por isso que a execução do algoritmo termina, mais cedo ou mais tarde.

Exercícios 4

1. ★ INVARIANTE. Mostre que no início de cada iteração de buscaBinaria (ou seja, imediatamente antes da comparação de x com $v[d]$) tem-se $v[e] < x \leq v[d]$.
2. VARIANTES DO ALGORITMO. Escreva uma versão da função buscaBinaria que tenha o seguinte invariante: no início de cada iteração, $v[e-1] < x \leq v[d]$. Repita com $v[e-1] < x \leq v[d+1]$. Repita com $v[e] < x \leq v[d+1]$.
3. VETORES GRANDES. Se o número de elementos do vetor estiver próximo de `INT_MAX`, o código da busca binária pode descarrilar, em virtude de um [overflow aritmético](#), ao calcular $m = (e + d)/2$. Como evitar isso?

Desempenho da busca binária

Quantas iterações a função buscaBinaria executa ao processar um vetor com n elementos? No início da primeira iteração, $d - e$ vale aproximadamente n . No início da segunda, vale aproximadamente $n/2$. No início da terceira, aproximadamente $n/4$. No início da $k+1$ -ésima, aproximadamente $n/2^k$. Quando k passar de $\log n$, o valor da expressão $n/2^k$ fica menor que 1 e a execução do algoritmo para. (Veja o exercício de cálculo de $\log n$ em [outra página](#).) Logo, o número de iterações é aproximadamente

$$\log n$$

tanto no pior caso quanto no melhor. O número aproximado de comparações de x com elementos do vetor também é $\log n$. Veja alguns exemplos quando n é potência de 2:

n	32	1024	32768	1048576	33554432	...
$\log n$	5	10	15	20	25	...

O valor de $\log n$ cresce muito devagar pois $\log$ transforma multiplicações em adições. Por exemplo, se a busca em um vetor de tamanho n exige C comparações, então a busca em um vetor de tamanho $2n$ exigirá apenas $C+1$ comparações, a busca em um vetor de tamanho $8n$ exigirá apenas $C+3$ comparações, e a busca em um vetor de tamanho $100n$ exigirá menos que $C+7$ comparações.

O consumo de tempo da função buscaBinaria é proporcional ao número de comparações de x com elementos de v . O consumo de tempo é, portanto, proporcional a $\log n$.

Exercícios 5

1. Suponha que o vetor v tem 511 elementos e que x não está no vetor. Quantas vezes, exatamente, a função buscaBinaria comparará x com um elemento do vetor? Suponha agora que o vetor v tem 50000 elementos e que x não está no vetor. Quantas vezes, aproximadamente, a função buscaBinaria comparará x com um elemento do vetor?
2. Se preciso de t segundos para fazer uma busca binária em um vetor de n elementos, de quanto tempo preciso para fazer uma busca em um vetor de n^2 elementos?
3. NÚMERO EXATO DE ITERAÇÕES. Mostre que, no pior caso, a função buscaBinaria faz exatamente $1 + \lg(n)$ comparações de x com elementos do vetor, sendo $\lg(n)$ o piso de $\log n$.

Exercícios 6

Implementações de busca binária exigem cuidado e atenção aos detalhes. É muito fácil escrever uma implementação que dá respostas erradas ou “entra em loop”. Os exercícios abaixo discutem versões alternativas da função buscaBinaria, diferentes da que examinamos acima. Todas procuram encontrar x em $v[0..n-1]$. Todas produzem (ou deveriam produzir) um índice j em $0..n$ tal que $v[j-1] < x \leq v[j]$.

1. Mostre que a seguinte versão de `buscaBinaria` está correta. Ela é quase tão bonita quanto a versão discutida acima.

```
e = 0; d = n;
while (e < d) { // v[e-1] < x <= v[d]
    m = (e + d)/2;
    if (v[m] < x) e = m+1;
    else d = m;
} // e == d
return d;
```

2. Mostre que a seguinte versão de `buscaBinaria` está correta. Acho que ela é um pouco mais feia que as versões anteriores.

```
e = 0; d = n-1;
while (e <= d) { // v[e-1] < x <= v[d+1]
    m = (e + d)/2;
    if (v[m] < x) e = m+1;
    else d = m-1; } // e == d+1
return d+1;
```

3. A seguinte versão de `buscaBinaria` está correta?

```
e = -1; d = n-1;
while (e < d) {
    m = (e + d)/2;
    if (v[m] < x) e = m;
    else d = m-1; }
return d+1;
```

4. A seguinte versão de `buscaBinaria` está correta?

```
e = -1; d = n-1;
while (e < d) {
    m = (e + d + 1)/2;
    if (v[m] < x) e = m;
    else d = m-1; }
return d+1;
```

Exercícios 7

- ★ Escreva uma implementação de busca binária que receba um inteiro x e um vetor crescente $v[0..n-1]$ e devolva j tal que $v[j-1] \leq x < v[j]$. Quais os possíveis valores de j ?
- Escreva uma implementação de busca binária que procure x num vetor crescente $v[0..n]$. Escreva uma implementação da busca binária que procure x em $v[1..n]$.
- Escreva uma implementação de busca binária que procure x num vetor *decrecente* $v[0..n-1]$.
- Modifique o código da função `buscaBinaria` de modo a resolver o problema de busca enunciado no [início deste capítulo](#).

Versão recursiva da busca binária

Para formular uma versão [recursiva](#) da busca binária é preciso generalizar ligeiramente o problema, trocando $v[0..n-1]$ por $v[a..b]$. A ponte entre a formulação básica e a generalizada é uma “função-embalagem” (= *wrapper-function*) `buscaBinaria2` que apenas repassa o serviço para uma função recursiva `bb`.

```
// Esta função recebe um vetor crescente
// v[0..n-1] e um inteiro x e devolve um
// índice j em 0..n tal que
```

```
// v[j-1] < x <= v[j].

int
buscaBinaria2 (int x, int n, int v[]) {
    return bb (x, -1, n, v);
}

// Recebe um vetor crescente v[e+1..d-1] e
// um inteiro x tal que v[e] < x <= v[d] e
// devolve um índice j em e+1..d tal que
// v[j-1] < x <= v[j].

static int
bb (int x, int e, int d, int v[]) {
    if (e == d-1) return d;
    else {
        int m = (e + d)/2;
        if (v[m] < x)
            return bb (x, m, d, v);
        else
            return bb (x, e, m, v);
    }
}
```

(A palavra-chave `static` está aí apenas para indicar que a função `bb` tem caráter auxiliar, e não deve ser invocada diretamente pelo usuário do algoritmo de busca binária.)

Qual a profundidade da recursão na função `bb`? Ou seja, quantas vezes `bb` chama a si mesma? Resposta: cerca de $\log n$ vezes.

Exercícios 8

1. CORREÇÃO. Prove que a função recursiva `bb` está correta. Observe que a condição $v[e] < x \leq v[d]$ garante que $e \leq d-1$. Verifique também que cada invocação de `bb` satisfaz as condições estabelecidas na documentação da função.
2. Discuta a seguinte versão alternativa da função `buscabinaria2`:

```
int buscaBinaria3 (int x, int n, int v[]) {
    if (v[n-1] < x) return n;
    if (x <= v[0]) return 0;
    return bb (x, 0, n-1, v); }
```

3. Discuta a seguinte versão alternativa da função `bb`. Escreva a documentação da função.

```
int bb2 (int x, int e, int d, int v[]) {
    if (e > d) return d+1;
    else {
        int m = (e + d)/2;
        if (v[m] < x)
            return bb2 (x, m+1, d, v);
        else
            return bb2 (x, e, m-1, v); } }
```

Exercícios 9

1. Two SUM. Escreva um algoritmo eficiente para o seguinte problema: dados um vetor crescente $v[0..n-1]$ de números inteiros e um número inteiro t , encontrar dois índices distintos i e j tais que $v[i] + v[j] = t$.

2. Escreva uma função que receba um vetor inteiro *estritamente* crescente $v[0..n-1]$ e devolva um índice i entre 0 e $n-1$ tal que $v[i] == i$ ou, se tal i não existe, devolve -1 . A função não deve fazer mais que $\log n$ comparações envolvendo elementos de v .
3. Escreva uma função eficiente que receba inteiros estritamente positivos k e n e calcule k^n . Quantas multiplicações sua função executa?
4. VETOR DE STRINGS. Suponha que $v[0..n-1]$ é um vetor de [strings ASCII](#) em [ordem lexicográfica](#). Escreva uma função que receba uma string x e devolva um índice j tal que $v[j-1] < x \leq v[j]$, onde “<” e “≤” se referem à ordem lexicográfica.
5. VETOR DE STRUCTS. Suponha que cada elemento do vetor $v[0..n-1]$ é uma [struct](#) com dois campos: o nome de um aluno e o número do aluno. Suponha que o vetor está em ordem crescente de números. Escreva uma função de busca binária que receba o número de um aluno e devolva o seu nome. Se o número não está no vetor, a função deve devolver a [string](#) vazia.
6. INTERVALOS DISJUNTOS. Suponha dado um conjunto de intervalos fechados disjuntos entre si. Suponha que os extremos dos intervalos são números inteiros. Escreva um programa que receba um inteiro como argumento e determine o intervalo a que o inteiro pertence. Por exemplo, se os intervalos são $1643..2033$, $5532..7643$, $8999..10332$, $5666653..5669321$, o número 9122 pertence ao terceiro intervalo e 8122 não pertence a intervalo algum.
7. Familiarize-se com a função `bsearch` da biblioteca [stdlib](#).

Veja o verbete [Binary search algorithm](#) na Wikipedia.

Veja também os capítulos 4 e 5 do [Programming Pearls](#).

Atualizado em 2018-09-15

<https://www.ime.usp.br/~pf/algoritmos/>

© Paulo Feofiloff

[DCC-IME-USP](#)